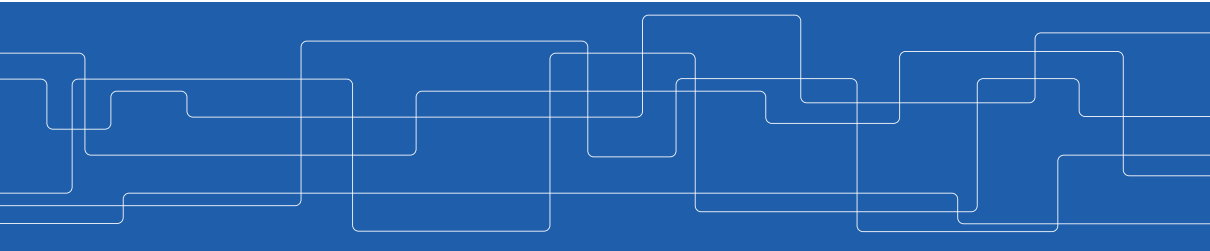


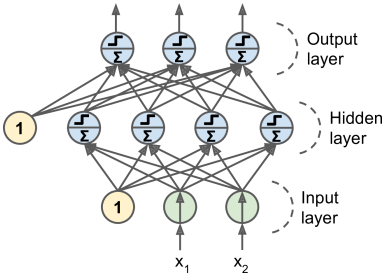

# Training Deep Feedforwards Networks

Amir H. Payberah
payberah@kth.se
2026-11-05

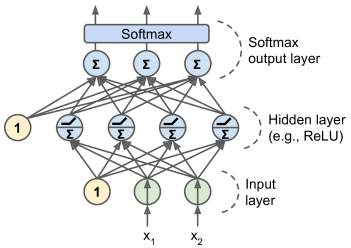


# Feedforward Neural Network Architecture

- A feedforward neural network is composed of:
  - One input layer
  - One or more hidden layers
  - One final output layer



# Feedforward Network in TensorFlow




```
n_output = 3
n_hidden = 4
n_features = 2

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_hidden, input_shape=(n_features,), activation="relu"))
model.add(keras.layers.Dense(n_output, activation="softmax"))

model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=30)
```

# Challenges of Training Feedforward Neural Networks

- Challenges ...



# Challenges of Training Feedforward Neural Networks

- Challenges ...
- Overfitting: risk of overfitting a model with large number of parameters.



# Challenges of Training Feedforward Neural Networks

- Challenges ...
- Overfitting: risk of overfitting a model with large number of parameters.
- Vanishing/exploding gradients: hard to train lower layers.



# Challenges of Training Feedforward Neural Networks

- Challenges ...
- Overfitting: risk of overfitting a model with large number of parameters.
- Vanishing/exploding gradients: hard to train lower layers.
- Training speed: slow training with large networks.



# Overfitting



# High Degree of Freedom and Overfitting Problem

- With large number of parameters, a network has a high degree of freedom.
- It can fit a huge variety of complex datasets.



# High Degree of Freedom and Overfitting Problem

- With large number of parameters, a network has a high degree of freedom.
- It can fit a huge variety of complex datasets.
- This flexibility also means that it is prone to overfitting on training set.



# High Degree of Freedom and Overfitting Problem

- With large number of parameters, a network has a high degree of freedom.
- It can fit a huge variety of complex datasets.
- This flexibility also means that it is prone to overfitting on training set.
- Let's reduce the degree of freedom a model.



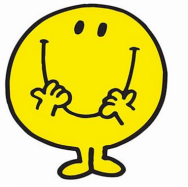
# Avoiding Overfitting

- Early stopping
- $l1$ and $l2$ regularization
- Dropout
- Data augmentation



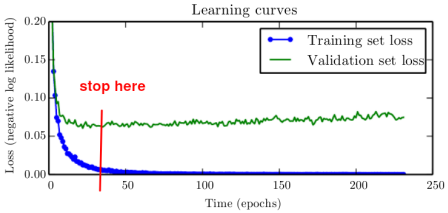
# Avoiding Overfitting

- Early stopping
- $l1$ and $l2$ regularization
- Dropout
- Data augmentation



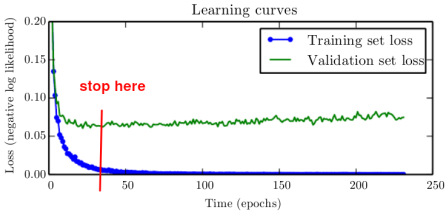
# Early Stopping (1/2)

- As the training steps go by, its prediction error on the training/validation set naturally goes down.



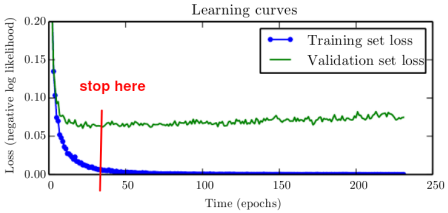
# Early Stopping (1/2)

- As the training steps go by, its prediction error on the training/validation set naturally goes down.
- After a while the validation error stops decreasing and starts to go back up.
  - The model has started to overfit the training data.



# Early Stopping (1/2)

- As the training steps go by, its prediction error on the training/validation set naturally goes down.
- After a while the validation error stops decreasing and starts to go back up.
  - The model has started to overfit the training data.
- In the early stopping, we stop training when the validation error reaches a minimum.






# Early Stopping (2/2)

```
from tensorflow.keras.callbacks import EarlyStopping

model = tf.keras.models.Sequential(...)

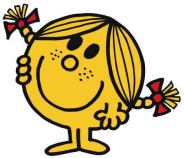
model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

earlystop_callback = EarlyStopping(monitor='accuracy', min_delta=0.05, patience=1)

model.fit(x_train, y_train, epochs=500, callbacks=[earlystop_callback])
```

# Avoiding Overfitting

- Early stopping
- $l1$ and $l2$ regularization
- Dropout
- Data augmentation





# $l1$ and $l2$ Regularization (1/3)

- Penalize large values of weights $\mathtt{w}_\mathtt{j}$.

$$\tilde{\mathtt{J}}(\mathbf{w}) = \mathtt{J}(\mathbf{w}) + \lambda \mathtt{R}(\mathbf{w})$$

- Two questions:
  1. How should we define $\mathtt{R}(\mathtt{w})$?
  2. How do we determine $\lambda$?



# /1 and /2 Regularization (2/3)

- /1 regression: $R(\mathbf{w}) = \sum_{i=1}^{n} |w_i|$ is added to the cost function.

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{i=1}^{n} |w_i|$$

```
keras.layers.Dense(100, activation="relu", kernel_regularizer=keras.regularizers.l1(0.1))
```

# $l1$ and $l2$ Regularization (3/3)

- $l2$ regression: $R(\mathbf{w}) = \sum_{i=1}^{n} w_i^2$ is added to the cost function.

$$\tilde{J}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{i=1}^{n} w_i^2$$

```
keras.layers.Dense(100, activation="relu", kernel_regularizer=keras.regularizers.l2(0.01))
```

# Avoiding Overfitting

- Early stopping
- /1 and /2 regularization
- Dropout
- Data augmentation



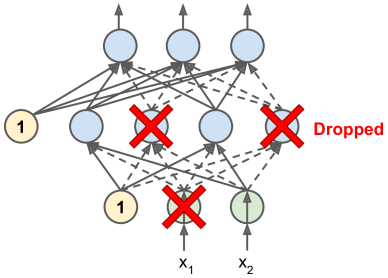
# Dropout (1/4)

- Would a company perform better if its employees were told to toss a coin every morning to decide whether or not to go to work?



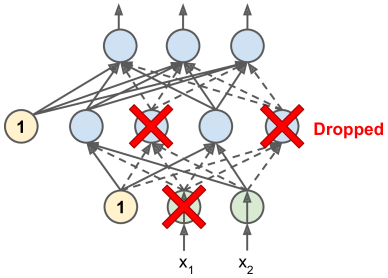
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.



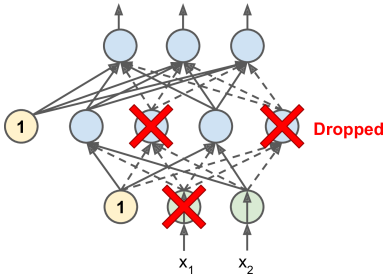
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.
  - The hyperparameter p is called the dropout rate.



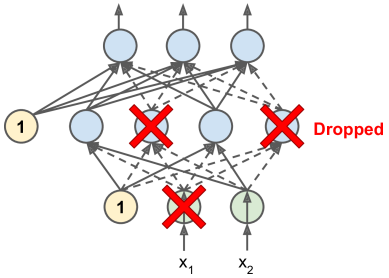
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.
  - The hyperparameter p is called the dropout rate.
  - A neuron will be entirely ignored during this training step.



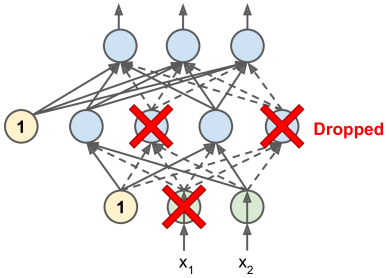
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.
  - The hyperparameter p is called the dropout rate.
  - A neuron will be entirely ignored during this training step.
  - It may be active during the next step.



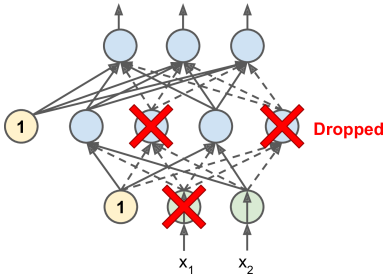
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.
  - The hyperparameter p is called the dropout rate.
  - A neuron will be entirely ignored during this training step.
  - It may be active during the next step.
  - Exclude the output neurons.



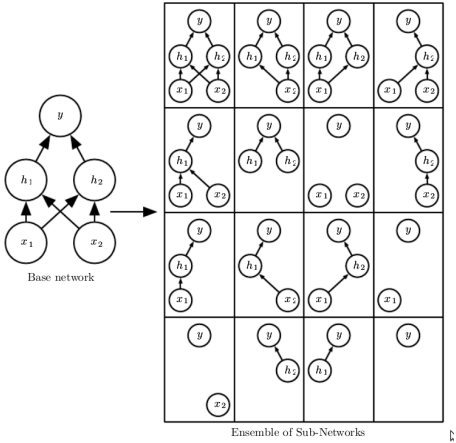
# Dropout (2/4)

- At each training step, each neuron drops out temporarily with a probability p.
  - The hyperparameter p is called the dropout rate.
  - A neuron will be entirely ignored during this training step.
  - It may be active during the next step.
  - Exclude the output neurons.
- After training, neurons don't get dropped anymore.



# Dropout (3/4)

- Each neuron can be either present or absent.
- $2^N$ possible networks, where $N$ is the total number of droppable neurons.
  - $N = 4$ in this figure.






# Dropout (4/4)

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
```

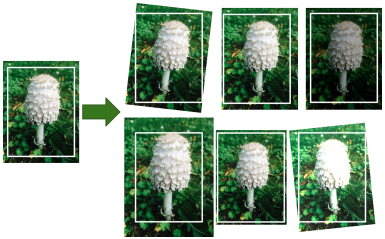
# Avoiding Overfitting

- Early stopping
- $l1$ and $l2$ regularization
- Dropout
- Data augmentation



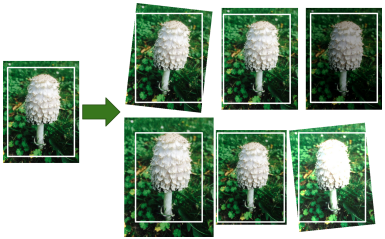
# Data Augmentation

- One way to make a model generalize better is to train it on more data.
- This will reduce overfitting.



# Data Augmentation

- One way to make a model generalize better is to train it on more data.
- This will reduce overfitting.
- Create fake data and add it to the training set.
  - E.g., in an image classification we can slightly shift, rotate and resize an image.
  - Add the resulting pictures to the training set.



# Vanishing/Exploding Gradients





# Vanishing/Exploding Gradients Problem (1/4)

- The backpropagation goes from output to input layer, and propagates the error gradient on the way.

$$\mathtt{w}^{(\mathtt{next})} = \mathtt{w} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}}$$



# Vanishing/Exploding Gradients Problem (1/4)

- The backpropagation goes from output to input layer, and propagates the error gradient on the way.

$$\mathtt{w}^{(\mathtt{next})} = \mathtt{w} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}}$$

- Gradients often get smaller and smaller as the algorithm progresses down to the lower layers.
- As a result, the gradient descent update leaves the lower layer connection weights virtually unchanged.

# Vanishing/Exploding Gradients Problem (1/4)

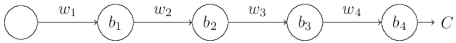
- The backpropagation goes from output to input layer, and propagates the error gradient on the way.

$$\mathtt{w}^{(\mathtt{next})} = \mathtt{w} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}}$$

- Gradients often get smaller and smaller as the algorithm progresses down to the lower layers.
- As a result, the gradient descent update leaves the lower layer connection weights virtually unchanged.
- This is called the vanishing gradients problem.

# Vanishing/Exploding Gradients Problem (2/4)

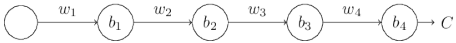
- Assume a network with just a single neuron in each layer.




  - $w_1, w_2, \cdots$ are the weights
  - $b_1, b_2, \cdots$ are the biases
  - $C$ is the cost function

# Vanishing/Exploding Gradients Problem (2/4)

- Assume a network with just a single neuron in each layer.




  - $w_1, w_2, \cdots$ are the weights
  - $b_1, b_2, \cdots$ are the biases
  - $C$ is the cost function

- The output $a_j$ from the $j$th neuron is $\sigma(z_j)$.
  - $\sigma$ is the sigmoid activation function
  - $z_j = w_j a_{j-1} + b_j$
  - E.g., $a_4 = \sigma(z_4) = \sigma(w_4 a_3 + b_4)$

# Vanishing/Exploding Gradients Problem (3/4)

- Lets compute the gradient associated to the first hidden neuron ($\frac{\partial \mathtt{C}}{\partial \mathtt{b}_1}$).




$$\frac{\partial \mathtt{C}}{\partial \mathtt{b}_1} = \frac{\partial \mathtt{C}}{\partial \mathtt{a}_4} \times \frac{\partial \mathtt{a}_4}{\partial \mathtt{z}_4} \times \frac{\partial \mathtt{z}_4}{\partial \mathtt{a}_3} \times \frac{\partial \mathtt{a}_3}{\partial \mathtt{z}_3} \times \frac{\partial \mathtt{z}_3}{\partial \mathtt{a}_2} \times \frac{\partial \mathtt{a}_2}{\partial \mathtt{z}_2} \times \frac{\partial \mathtt{z}_2}{\partial \mathtt{a}_1} \times \frac{\partial \mathtt{a}_1}{\partial \mathtt{z}_1} \times \frac{\partial \mathtt{z}_1}{\partial \mathtt{b}_1}$$

# Vanishing/Exploding Gradients Problem (3/4)

- Lets compute the gradient associated to the first hidden neuron ($\frac{\partial C}{\partial b_1}$).




$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times \frac{\partial z_4}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial b_1}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times \frac{\partial w_4 a_3 + b_4}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial w_3 a_2 + b_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial w_2 a_1 + b_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial w_1 a_0 + b_1}{\partial b_1}$$

# Vanishing/Exploding Gradients Problem (3/4)

- Lets compute the gradient associated to the first hidden neuron ($\frac{\partial C}{\partial b_1}$).




$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times \frac{\partial z_4}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial b_1}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times \frac{\partial w_4 a_3 + b_4}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial w_3 a_2 + b_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial w_2 a_1 + b_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial w_1 a_0 + b_1}{\partial b_1}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times w_4 \times \frac{\partial a_3}{\partial z_3} \times w_3 \times \frac{\partial a_2}{\partial z_2} \times w_2 \times \frac{\partial a_1}{\partial z_1} \times 1$$

# Vanishing/Exploding Gradients Problem (4/4)

- Cocnsider $\frac{\partial \mathtt{C}}{\partial \mathtt{b}_3}$.




$$\frac{\partial \mathtt{C}}{\partial \mathtt{b}_3} = \frac{\partial \mathtt{C}}{\partial \mathtt{a}_4} \times \frac{\partial \mathtt{a}_4}{\partial \mathtt{z}_4} \times \mathtt{w}_4 \times \frac{\partial \mathtt{a}_3}{\partial \mathtt{z}_3}$$

# Vanishing/Exploding Gradients Problem (4/4)

- Cocnsider $\frac{\partial C}{\partial b_3}$.




$$\frac{\partial C}{\partial b_3} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times w_4 \times \frac{\partial a_3}{\partial z_3}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times w_4 \times \frac{\partial a_3}{\partial z_3} \times w_3 \times \frac{\partial a_2}{\partial z_2} \times w_2 \times \frac{\partial a_1}{\partial z_1} \times 1$$

# Vanishing/Exploding Gradients Problem (4/4)

- Cocnsider $\frac{\partial C}{\partial b_3}$.




$$\frac{\partial C}{\partial b_3} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times w_4 \times \frac{\partial a_3}{\partial z_3}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \times w_4 \times \frac{\partial a_3}{\partial z_3} \times w_3 \times \frac{\partial a_2}{\partial z_2} \times w_2 \times \frac{\partial a_1}{\partial z_1} \times 1$$

- Assume $w_3 \times \frac{\partial a_2}{\partial z_2} < \frac{1}{4}$ and $w_2 \times \frac{\partial a_1}{\partial z_1} < \frac{1}{4}$
  - The gradient $\frac{\partial C}{\partial b_1}$ be a factor of 16 (or more) smaller than $\frac{\partial C}{\partial b_3}$.
  - This is the essential origin of the vanishing gradient problem.

# Overcoming the Vanishing Gradient

- Parameter initialization strategies
- Nonsaturating activation function
- Batch normalization
- Gradient clipping



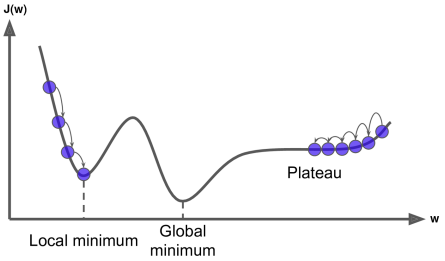
# Overcoming the Vanishing Gradient

- Parameter initiazlization strategies
- Nonsaturating activation function
- Batch normalization
- Gradient clipping



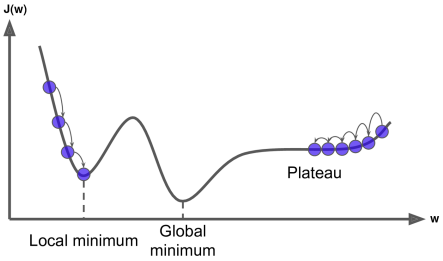
# Parameter Initialization Strategies (1/2)

- The non-linearity of a neural network causes the cost functions to become non-convex.



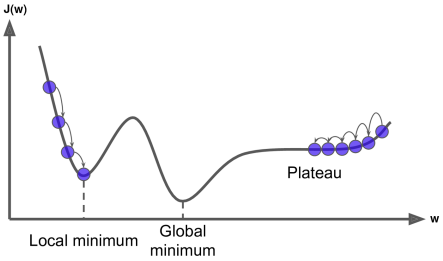
# Parameter Initialization Strategies (1/2)

- The non-linearity of a neural network causes the cost functions to become non-convex.
- The stochastic gradient descent on non-convex cost functions performs is sensitive to the values of the initial parameters.



# Parameter Initialization Strategies (1/2)

- The non-linearity of a neural network causes the cost functions to become non-convex.
- The stochastic gradient descent on non-convex cost functions performs is sensitive to the values of the initial parameters.
- Designing initialization strategies is a difficult task.





# Parameter Initialization Strategies (2/2)

- The initial parameters need to break symmetry between different units.



# Parameter Initialization Strategies (2/2)

- The initial parameters need to break symmetry between different units.
- We need the signals to flow properly in both directions.



# Parameter Initialization Strategies (2/2)

- The initial parameters need to break symmetry between different units.
- We need the signals to flow properly in both directions.
- Glorot initialization, for none, logistic, sigmoid, and tanh.



# Parameter Initialization Strategies (2/2)

- The initial parameters need to break symmetry between different units.
- We need the signals to flow properly in both directions.
- Glorot initialization, for none, logistic, sigmoid, and tanh.
- He initialization, for ReLU.



# Parameter Initialization Strategies (2/2)

- The initial parameters need to break symmetry between different units.
- We need the signals to flow properly in both directions.
- Glorot initialization, for none, logistic, sigmoid, and tanh.
- He initialization, for ReLU.

```
keras.layers.Dense(10, activation="relu", kernel_initializer="he_normal")
```

# Overcoming the Vanishing Gradient

- Parameter initiazlization strategies
- Nonsaturating activation function
- Batch normalization
- Gradient clipping





# Nonsaturating Activation Functions (1/2)

- $\texttt{ReLU}(z) = \max(0, z)$
- The dying ReLUs problem.

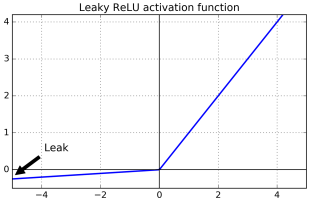


# Nonsaturating Activation Functions (1/2)

- $\texttt{ReLU}(z) = \max(0, z)$
- The dying ReLUs problem.
  - During training, some neurons stop outputting anything other than 0.
  - E.g., when the weighted sum of the neuron's inputs is negative, it starts outputting 0.

# Nonsaturating Activation Functions (1/2)

- $\texttt{ReLU}(z) = \max(0, z)$
- The dying ReLUs problem.
  - During training, some neurons stop outputting anything other than 0.
  - E.g., when the weighted sum of the neuron's inputs is negative, it starts outputting 0.
- Use leaky ReLU instead: $\texttt{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$.
  - $\alpha$ is the slope of the function for $z < 0$.





# Nonsaturating Activation Functions (2/2)

- Randomized Leaky ReLU (RReLU)
  - $\alpha$ is picked randomly during training, and it is fixed during testing.



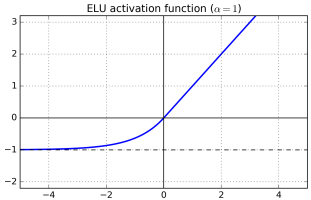
# Nonsaturating Activation Functions (2/2)

- Randomized Leaky ReLU (RReLU)
  - $\alpha$ is picked randomly during training, and it is fixed during testing.
- Parametric Leaky ReLU (PReLU)
  - Learn $\alpha$ during training (instead of being a hyperparameter).

# Nonsaturating Activation Functions (2/2)

- Randomized Leaky ReLU (RReLU)
  - $\alpha$ is picked randomly during training, and it is fixed during testing.
- Parametric Leaky ReLU (PReLU)
  - Learn $\alpha$ during training (instead of being a hyperparameter).
- Exponential Linear Unit (ELU)

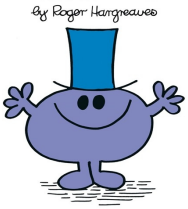
$$\mathrm{ELU}_\alpha(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$




```
# elu
keras.layers.Dense(10, activation="elu")
```

# Overcoming the Vanishing Gradient

- Parameter initiazlization strategies
- Nonsaturating activation function
- Batch normalization
- Gradient clipping





# Batch Normalization (1/4)

- When a network trains, the values going into each layer can change a lot.



# Batch Normalization (1/4)

- When a network trains, the values going into each layer can change a lot.
- Batch Normalization fixes this by keeping these values in a consistent range.



# Batch Normalization (1/4)

- When a network trains, the values going into each layer can change a lot.
- Batch Normalization fixes this by keeping these values in a consistent range.
- Normalizing the outputs of each layer (so they have mean $\approx 0$ and variance $\approx 1$ within a mini-batch).



# Batch Normalization (1/4)

- When a network trains, the values going into each layer can change a lot.
- Batch Normalization fixes this by keeping these values in a consistent range.
- Normalizing the outputs of each layer (so they have mean $\approx 0$ and variance $\approx 1$ within a mini-batch).
- Then scaling and shifting them with learnable parameters.



# Batch Normalization (2/4)

- It's zero-centering and normalizing the inputs, then scaling and shifting the result.

# Batch Normalization (2/4)

- It's zero-centering and normalizing the inputs, then scaling and shifting the result.

$$\mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

$$\sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (\mathbf{x}^{(i)} - \mu_B)^2$$

- $\mu_B$: the empirical mean, evaluated over the whole mini-batch B.
- $\sigma_B$: the empirical standard deviation, also evaluated over the whole mini-batch.
- $m_B$: the number of instances in the mini-batch.

# Batch Normalization (3/4)

$$\hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$
$$\mathbf{z}^{(i)} = \gamma \hat{\mathbf{x}}^{(i)} + \beta$$

- $\hat{\mathbf{x}}^{(i)}$: the zero-centered and normalized input.
- $\mathbf{z}^{(i)}$: the output of the BN operation, which is a scaled and shifted version of the inputs.
- $\gamma$: the scaling parameter vector for the layer.
- $\beta$: the shifting parameter (offset) vector for the layer.
- $\epsilon$: a tiny number to avoid division by zero.
- $\otimes$: represents the element-wise multiplication.




# Batch Normalization (4/4)

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation="softmax")
])
```

# Overcoming the Vanishing Gradient

- Parameter initiazlization strategies
- Nonsaturating activation function
- Batch normalization
- Gradient clipping





# Gradient Clipping

- Gradient clipping: clip the gradients during backpropagation so that they never exceed some threshold.

```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```



# Gradient Clipping

- Gradient clipping: clip the gradients during backpropagation so that they never exceed some threshold.

```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

- Setting the `clipvalue` or `clipnorm` argument when creating an optimizer.



# Gradient Clipping

- Gradient clipping: clip the gradients during backpropagation so that they never exceed some threshold.

```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

- Setting the `clipvalue` or `clipnorm` argument when creating an optimizer.
- `clipvalue=1.0` and `clipnorm=1.0`: values between -1.0 and 1.0.



# Gradient Clipping

- Gradient clipping: clip the gradients during backpropagation so that they never exceed some threshold.

```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

- Setting the `clipvalue` or `clipnorm` argument when creating an optimizer.
- `clipvalue=1.0` and `clipnorm=1.0`: values between -1.0 and 1.0.
- `clipvalue=1.0`: $[0.9, 100.0] \Rightarrow [0.9, 1.0]$



# Gradient Clipping

- Gradient clipping: clip the gradients during backpropagation so that they never exceed some threshold.

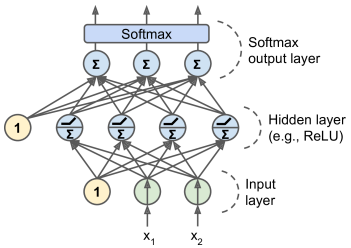
```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

- Setting the `clipvalue` or `clipnorm` argument when creating an optimizer.
- `clipvalue=1.0` and `clipnorm=1.0`: values between -1.0 and 1.0.
- `clipvalue=1.0`: $[0.9, 100.0] \Rightarrow [0.9, 1.0]$
- `clipnorm=1.0`: $[0.9, 100.0] \Rightarrow [0.00899964, 0.9999595]$

# Training Speed



# Regular Gradient Descent Optimization (1/2)




```
n_output = 3
n_hidden = 4
n_features = 2

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_hidden, input_shape=(n_features,), activation="relu"))
model.add(keras.layers.Dense(n_output, activation="softmax"))

model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=30)
```



# Regular Gradient Descent Optimization (2/2)

- Gradient descent optimization algorithm
- It updates the weights $\mathtt{w}_{\mathtt{i}}^{(\mathtt{next})} = \mathtt{w}_{\mathtt{i}} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_{\mathtt{i}}}$
- Better optimization algorithms to improve the training speed

# Optimization Algorithms

- Momentum
- RMSProp
- Adam Optimization



# Optimization Algorithms

- Momentum
- RMSProp
- Adam optimization



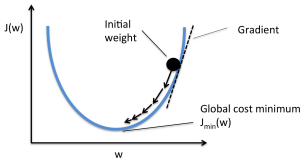


# Momentum (1/2)

- We can see the change in the parameters **w** as motion: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$

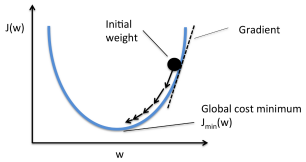
# Momentum (1/2)

- We can see the change in the parameters **w** as motion: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$



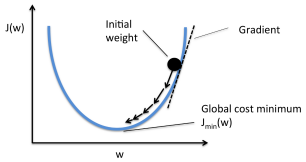
# Momentum (1/2)

- We can see the change in the parameters $\mathbf{w}$ as motion: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$
- Giving the update process a tendency to keep moving in the same direction.



# Momentum (1/2)

- We can see the change in the parameters **w** as motion: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$
- Giving the update process a tendency to keep moving in the same direction.
- It can help to escape from bad local minima pits.





# Momentum (2/2)

- Regular gradient descent optimization: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$



# Momentum (2/2)

- Regular gradient descent optimization: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$
- Momentum optimization cares about what previous gradients were.

# Momentum (2/2)

- Regular gradient descent optimization: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$
- Momentum optimization cares about what previous gradients were.
- At each iteration, it adds the local gradient to the momentum vector **m**.

$$\mathtt{m}_\mathtt{i} = \beta \mathtt{m}_\mathtt{i} + \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$$

$$\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \mathtt{m}_\mathtt{i}$$

# Momentum (2/2)

- Regular gradient descent optimization: $\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$
- Momentum optimization cares about what previous gradients were.
- At each iteration, it adds the local gradient to the momentum vector **m**.

$$\mathtt{m}_\mathtt{i} = \beta \mathtt{m}_\mathtt{i} + \eta \frac{\partial \mathtt{J}(\mathbf{w})}{\partial \mathtt{w}_\mathtt{i}}$$

$$\mathtt{w}_\mathtt{i}^{(\mathtt{next})} = \mathtt{w}_\mathtt{i} - \mathtt{m}_\mathtt{i}$$

- $\beta$ is called momentum, ans it is between 0 and 1.

# Momentum (2/2)

- Regular gradient descent optimization: $w_i^{(next)} = w_i - \eta \frac{\partial J(\mathbf{w})}{\partial w_i}$
- Momentum optimization cares about what previous gradients were.
- At each iteration, it adds the local gradient to the momentum vector **m**.

$$m_i = \beta m_i + \eta \frac{\partial J(\mathbf{w})}{\partial w_i}$$
$$w_i^{(next)} = w_i - m_i$$

- $\beta$ is called momentum, ans it is between 0 and 1.

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
```

# Optimization Algorithms

- Momentum
- RMSProp
- Adam optimization






# RMSProp (1/2)

- It keeps track of a learning rate for each parameter.
- Adapts the learning rate over time (adaptive learning rate).




# RMSProp (1/2)

- It keeps track of a learning rate for each parameter.
- Adapts the learning rate over time (adaptive learning rate).
- Accumulates only the gradients from the most recent iterations.

# RMSProp (2/2)

- For each feature $\mathtt{w_i}$, we do the following steps:

$$\mathtt{s_i} = \beta \mathtt{s_i} + (1-\beta)\left(\frac{\partial \mathtt{J(w)}}{\partial \mathtt{w_i}}\right)^2$$

$$\mathtt{w_i^{(next)}} = \mathtt{w_i} - \frac{\eta}{\sqrt{\mathtt{s_i} + \epsilon}} \frac{\partial \mathtt{J(w)}}{\partial \mathtt{w_i}}$$

# RMSProp (2/2)

- For each feature $\mathtt{w_i}$, we do the following steps:

$$\mathtt{s_i} = \beta \mathtt{s_i} + (1-\beta)\left(\frac{\partial \mathtt{J(w)}}{\partial \mathtt{w_i}}\right)^2$$

$$\mathtt{w_i^{(next)}} = \mathtt{w_i} - \frac{\eta}{\sqrt{\mathtt{s_i} + \epsilon}} \frac{\partial \mathtt{J(w)}}{\partial \mathtt{w_i}}$$

```
optimizer = keras.optimizers.RMSprop(lr=0.001, rho=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
```

# Optimization Algorithms

- Momentum
- Nesterov momentum
- Adam optimization





# Adam Optimization (1/2)

- It combines the ideas of Momentum optimization and RMSProp.




# Adam Optimization (1/2)

- It combines the ideas of Momentum optimization and RMSProp.
- Adam = Momentum + RMSProp



# Adam Optimization (1/2)

- It combines the ideas of Momentum optimization and RMSProp.
- Adam = Momentum + RMSProp
- Momentum optimization: keeps track of the direction the gradients have been moving, so it smooths out noisy updates and moves faster in the right direction.



# Adam Optimization (1/2)

- It combines the ideas of Momentum optimization and RMSProp.
- Adam = Momentum + RMSProp
- Momentum optimization: keeps track of the direction the gradients have been moving, so it smooths out noisy updates and moves faster in the right direction.
- RMSProp: keeps track of how large or small each gradient has been, and adjusts the learning rate for each parameter so it doesn't overshoot or move too slowly.




# Adam Optimization (2/2)

```
optimizer = keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
```




# Summary

# Summary

- Overfitting
  - Early stopping, $l1$ and $l2$ regularization
  - Dropout, data augmentation
- Vanishing gradient
  - Parameter initialization, nonsaturating activation functions
  - Batch normalization, gradient clipping
- Training speed
  - Momentum, RMSProp, Adam optimization





# Reference

- Hands-on machine learning with Scikit-Learn and TensorFlow (Ch. 11)




# Questions?